

2022-01-0132 Published 29 Mar 2022



UDS Security Access for Constrained ECUs

Martin Thompson ZF Automotive UK Ltd

Citation: Thompson, M., "UDS Security Access for Constrained ECUs," SAE Technical Paper 2022-01-0132, 2022, doi:10.4271/2022-01-0132.

Received: 24 Jan 2022

Revised: 24 Jan 2022

Accepted: 13 Jan 2022

Abstract

Legacy electronic control units are, nowadays, required to implement cybersecurity measures, but they often do not have all the elements that are necessary to realize industry-standard cybersecurity controls. For example, they may not have hardware cryptographic accelerators, segregated areas of memory for storing keys, or one-time programmable memory areas. Such systems must still be protected with a sufficient level of rigor against attackers who wish to modify their operation or extract confidential information from them. A critical interface to defend is the Unified Diagnostics Service (UDS) interface which is used in many areas across the whole vehicle lifecycle. While the UDS service \$27 (Security Access) has a reputation for poor cybersecurity, there is nothing inherent in the way it operates which prevents a secure access-control from being implemented. This paper describes an approach to providing UDS

Security Access within systems which have very constrained processors (in terms of processing power, memory size and, in particular, cybersecurity features) which can be applied to multiple vehicles across many manufacturers. It describes, in detail, methods for generating UDS-Seeds and UDS-Keys in the absence of a hardware security module (HSM) with a true-random number generator, and without use (by the user who is requesting access) of IT-infrastructure. In addition, the problem of key-management and distribution is tackled head-on and not left as an implementation detail. A threat analysis has been performed (according to ISO/SAE 21434) using model-based tools, the results of which are presented in this paper. The constraints (some of which make it difficult to properly secure certain key material) result in risks which become clear in the threat analysis. Potential future users of this scheme can use this analysis to assess the residual risks in their own applications.

Introduction

Electronic control units on vehicles provide interfaces which can modify the content ECU in ways which may compromise the safety, emissions or other critical behaviors of the vehicle. As vehicles develop, in particular to future hybrid powertrains, the breadth of these functions will only get greater [1]. It is clear that such access must be controlled to allow only authorized users to execute these diagnostic functions [2].

The Unified Diagnostics Services (UDS) protocol stack is used to provide networked access to diagnostic services on all modern Electronics Control Units (ECUs). It can operate using a variety of physical interfaces (for example, CAN, Ethernet) but the fundamental protocol remains the same. UDS is documented within the ISO standard documents in the ISO14229 series. UDS Security Access (Service \$27) is intended to restrict "access for security, emissions, or safety reasons" [2]. It provides a standard challenge/response structure - the actual implementation (i.e. the form of the challenge and the response) is not specified. In the 2020 revision of the ISO14229 specification, UDS Authenticated Access (Service \$29) was added, which standardizes certificate-based asymmetric authentication methods, but places larger demands relating to the infrastructure required to authenticate users.

UDS provides multiple "levels" of Security Access. At the base level, anyone with access to the tools (which are publicly available) can gain access to the ECU. This base level does not provide any security, so only a small set of functions should be provided at this level. When other security levels are activated, only one can be active at any given time. The "meaning" of the levels of security access, and therefore which privileges are enabled, is embedded within the software of the ECU.

When legacy ECUs units are required to implement cybersecurity measures, they often do not have all the facilities that would be expected and are necessary to implement industry-standard cybersecurity controls.

This paper describes a design-pattern by which legacy control units can provide a reasonable level of UDS-based cybersecurity, working within the constraints often encountered within such systems. We present these constraints, and a realistic threat model which is applied. A detailed description of the system and notes on the implementation are also presented. This is followed by a summary of the threat analysis that has been performed. It should be noted that there exists a tension between enabling legitimate mechanics access to potentially harmful functions (for example, those covered by 'right-to-repair' legislation, amongst others) and stopping attackers from doing damage. This paper does not offer a

solution to the problem of differentiating between legitimate and illegitimate individuals.

Note that this design-pattern is bounded - it considers the security of the UDS services. It does not provide defences against other potential vulnerabilities such as runtime message spoofing or the integrity of flash downloads, which can be mitigated using other techniques if this is necessary in any particular application.

There are no novel cryptographic elements or protocols described here. This paper provides a practical distillation of the most appropriate best practices, given a set of real-world constraints, from the existing cybersecurity literature and gathers them into a single, immediately applicable, design.

A Note on Terminology

This design-pattern is very similar to standard challenge/response authentication patterns already widely used in IT security [3, 4]. However, the challenge-response pair is described (for historical reasons) by ISO14229 as a “Seed&Key pair”. This confuses two meanings of the words Seed and Key which have well-defined meanings in the cybersecurity literature. In this document, we will use the word ‘key’ to mean a true cryptographic key of some kind, and ‘seed’ to mean the initialization value for a random number generator. For the UDS challenge, we will use the phrase “UDS-Seed” and correspondingly “UDS-Key” for the UDS response. These terms are summarized in Table 1.

There is also a device which is termed “the tester” - this is the device which is used by the mechanic to perform their diagnostic tasks. It is assumed that anyone who has one of these devices is an authorized user.

Constraints

A number of constraints have been considered when developing the UDS Security Access protocol documented in this paper. They are presented in Table 2, along with a justification. The constraints are taken from real-world projects with real vehicle-manufacturers (VMs).

Threat Model

This system is designed to provide a reasonable level of security against an attacker who has:

- Access to one or more ECUs.
- Access to a potentially large number of previously used UDS-seeds and the corresponding UDS-keys

TABLE 1 Terminology cross-reference

ISO14229 term	Traditional cybersecurity term	This document
Seed	Challenge	UDS-Seed
Key	Response	UDS-Key

TABLE 2 Constraints

Constraint	Justification
Must use \$27 security access	VMs are already familiar with this. Authentication service \$29 is novel.
Must use a single algorithm for all VMs and all security levels	To reduce the engineering effort involved in developing ECUs for multiple VMs, having a single algorithm is important. In addition, there is very limited code space in the target ECUs which provides additional incentive to avoid the use of multiple algorithms.
Tune the algorithm parameters to each VM	In order to provide cybersecurity, which is not shared between VMs, there must be some parameters relating to the algorithm which can be easily “tuned” to provide this.
There must be no use of back-end IT for the unlock process.	In many projects, VMs expect that the unlock process can be performed in isolation with the authentication provided through the “tester”.
There is no HSM in the target ECUs	This means that there is no acceleration for any cryptography, nor isolation of key material from application code.
An acceptably good source of entropy must be created within the ECU	There is no True Random Number Generator in the target ECUs.
Avoid introducing additional cryptographic primitives - re-use those which are already being used in other areas of the cybersecurity provision of the ECU (for example, securing code downloads)	In order to minimise the code-space overhead and development effort

It is **not** designed to defend against skilled attackers who can gain access to the authentication software which is used to map a UDS-Seed to a UDS-Key. This software is provided, within the tester, to the authorised mechanics who work on the target ECUs. It is expected that attackers who are able to obtain said software will be able to extract both the algorithm and the cryptographic key material from it. Publication of this knowledge would then allow unskilled attackers to bypass the protection.

Advantage is taken of the fact that there are opportunities for differing privilege levels. VMs who integrate the ECU within their vehicles are not allowed to download software or perform safety-critical tuning operations, so the impact of a leak of their key material is limited. The ECU producer’s key material can (and must) be better secured (as it has higher impact damage scenarios associated with it), but this is easier to achieve.

Setup

This paper covers the lifecycle of the assets required to perform the UDS unlock services. The various lifecycle phases are discussed in this section.

ECU Development

During the ECU development phase, the asymmetric keypairs used to defend each of the security levels are securely generated. For each security level, there is a private key (which will be provisioned into a tester device) and a public key, which is provisioned into the ECUs.

The areas of flash memory into which the keys will be provisioned are reserved in the ECU and set to a blank value which can be overwritten during production.

Another public/private keypair is created to control the access to the UDS services which will be used to provision the other keys. The public key goes into the ECU (via the software binary) and the private key to the production line. This forms a “key-provisioning-key”.

ECU Manufacture

During production, the correct keys for the destination of the ECU (the “verification-keys”) will be provisioned using UDS services. The production line uses the key-provisioning-key to authenticate to the ECU and provision the correct field keys.

The production line also checks a hash of all the field-keys provisioned to ensure the process has been successful and then informs the ECU to lock any unused key areas.

Tester Manufacture

There are at least 2 different testers in the system, one for the ECU-supplier and one for each VM. Each manufacturing line for the testers receives only the private keys appropriate to the specific testers that it is manufacturing, and not the other private keys.

Field Usage

In the field (i.e. once the ECU has been integrated into a vehicle and left the custody of the vehicle manufacturer - during the operational lifetime of the ECU), the user makes use of the tester to perform an authentication transaction with the ECU. The private key is used to sign a challenge and that signature is checked by the ECU using the public key. The VM tester is only authorized to reconfigure the non-safety critical parameters of the ECU. This allows the VM to tune parameters which are of interest, without affecting the safety of the ECU. Only the ECU-supplier tester is authorized to modify safety-critical aspects of the ECU. The authorization is controlled by the fact that these two types of tester are separate items and only contain the private keys which authenticate the activities which are allowed for the user that they belong to. Even if the same ECU is used in multiple different vehicles, belonging to multiple VMs, the use of different keys means that each VM

is only authorized to modify the ECUs for which they have the correct private key. In this way, authorization levels (between ECU supplier and each of the individual VMs) are controlled within an individual ECU design, even though the algorithm used is the same everywhere.

Design

The standard UDS Security Access process is followed, where the mechanic, who has possession of a “tester”, uses this to request access to the relevant security level, using the *RequestSeed* service. The ECU provides a UDS-Seed to the tester, which calculates the UDS-Key and returns it to the ECU. The ECU checks the UDS-Key and, if it is valid, enables the appropriate security level. This is shown in [Figure 1](#).

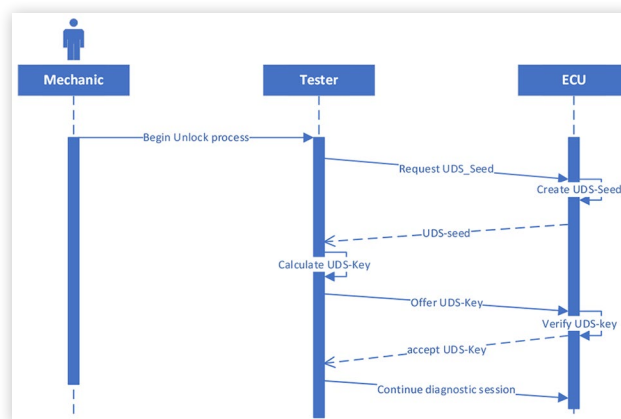
There are 3 critical processes within the ECU and the tester:

- Creating the UDS-Seed
- Generating the UDS-Key to return to the ECU
- Verifying the UDS-Key which is given to the ECU by the tester

The UDS-Seed is mapped to the UDS-Key using an RSA sign/verify operation with a private/public keypair. The private part is stored in the tester and used for signature (to prove that an authorised user is making the request). The public part is stored in the ECU and used for verification. The key-pair used for the signature generation and verification forms the single parameter which is used to differentiate accesses between different security levels and different VMs. There must be a different key-pair for each UDS security level, and each VM has a different set of key-pairs.

In the following, the private and public keys are referred to as the “signature-key-for-requested-security-level” and “verification-key-for-requested-security-level”.

FIGURE 1 The standard UDS Security Access Handshake



Creating the UDS Seed

Standard practice would be to generate this seed from a Cryptographically Strong Pseudo Random Number Generator (CSPRNG) seeded from a strong True Random Number Generator (TRNG). We still use a CSPRNG, but we do not have easy access to a source of good entropy (such as a TRNG) to seed it from, so an alternative means is described below.

Whenever a UDS-Seed is required, 244-bits of data are taken from the CSPRNG and the security level byte is appended in the lowest byte to form a 256-bit UDS-seed.

Entropy Source As there is no strong source of entropy (which cannot be manipulated by an attacker), we combine several sources of (potentially attacker-controlled) entropy to make the job of the attacker in controlling this as difficult as possible. In particular, we require the attacker to open the ECU to influence some of the sources of entropy and we link the entropy to the specific instance of the ECU being accessed.

At ECU start-up, entropy is gathered from sources such as:

- All ADC channels within the ECU (even those not used in the application) - each channel is sampled 1000 times after start-up. This limits the attacker's ability to influence the random number generator as this requires them to physically open up the ECU, which is an additional barrier to attack.
- The ECU configuration - the whole EEPROM/Data FLASH area which contains ECU-specific information. The use of ECU-specific information (for example, serial numbers, specific tune-data) ensures that the seed is different for every ECU.

These elements are fed into a SHA-256 [5] operation to form a single 256-bit entropy value which can be used to initialize a CSPRNG (see below).

Note that it would be possible to continuously sample entropy from the ECU during runtime and add this to the CSPRNG, but this adds complexity to the product which (in this design) is considered a poor trade-off relative to the small benefits offered.

CSPRNG To create the CSPRNG, the Hash-DRBG [6] construction is used, with SHA-256 used as the hash function.

Generating the UDS-Key to Return

In the tester, the UDS-Key is calculated with: $\text{UDS-key} = \text{Sign}(\text{signature-key-for-requested-security-level}, \text{UDS-seed})$

Verifying the Signature In the ECU, UDS-Seed is treated as the message and the UDS-Key is a signature. This combination is verified using: $\text{Verified} = \text{Verify}(\text{verification-key-for-requested-security-level}, \text{UDS-Seed}, \text{UDS-key})$

Key Management

In order to allow these services to be performed, a number of keys must be managed within various functions of the company. This covers IT, engineering and manufacturing.

The private keys used to sign the UDS-Seed are critical assets, as anyone with access to a specific private key can access the security level which is associated with that key. These assets live in 3 places: the engineering function, the tester manufacturing function, the tester-units themselves.

The private keys are protected in transit and at rest (against tampering and snooping) using authenticated AES encryption using a 256-bit key. A Windows Bitlocker-protected [7] virtual disk image is one option which is widely available in the target environments and is reasonably user-friendly - other technologies are of course available. The shared-key for this image is called the "key-encryption-key". The key-encryption-key must be generated from a high-quality password (ideally generated at random) and should be stored only in non-electronic form - of course ensuring there are enough copies for disaster recovery!

The private keys should not be stored in a cloud service or other networked computer in their unencrypted form. Ideally, operations using them should only be performed with workstations that are not network-connected. This limits the opportunity for a remote attacker to obtain the keys. The private key operations should be performed in an environment where 2 people are operating together (the "4-eyes" principle [8]).

The testers only contain those keys which are relevant to the specific intended VM. This ensures that they can only scale their own access to different ECUs, and not elevate to other privilege levels or VMs that they should not have access to. Access to testers containing the private keys used to authorize access to the most critical functions (which are normally reserved for the ECU design entity) must be strictly limited to those who need access.

Implementation Notes

Response to Repeated Requests

When the user makes repeated requests for a UDS-Seed, the same UDS-Seed shall be provided on each occasion, until a UDS-Key has been given in response. This prevents an attacker requesting UDS-Seeds until they receive one for which they know the correct UDS-Key.

Timing Sanctions

In order to cover interruptions to the UDS unlocking process, an appropriate time-out needs to be implemented. Upon transmission of a UDS-Seed value, the software shall consider that UDS-Seed value to be valid for a duration of (for example)

5 seconds only. After this duration, the UDS-Seed shall be considered invalid and the ECU will respond to a provided UDS-Key with a *conditionsNotCorrect* code.

The ECU tracks the number of incorrect attempts to unlock using a non-volatile variable *incorrect_UDS_keys_received*, which is managed as follows:

The variable *incorrect_UDS_keys_received*

- is set to zero when a correct UDS-key is received. It is initialized to zero for a new ECU.
- is incremented by 1 when an incorrect UDS-key is received.
- must be stored to NVM before any response is returned.
- is initialised from NVM at startup.

The variable *Delay_Timer* has an exponential dependence on *incorrect_UDS_keys_received*:

$$\text{Delay_Timer} = 2^{(\text{incorrect_UDS_keys_received}+1)}$$

The *Delay_Timer* is started on ECU startup, so that even the very first access will be subject to it. This means that the first UDS unlock can only be submitted after four seconds, providing a level of rate-limiting right from the start. Even if an attacker somehow prevents the update of *incorrect_UDS_keys_received* they can only attempt one UDS-Seed every four seconds.

Lockout

After more than two incorrect UDS-keys have been received, since power up, the ECU should no longer check if the UDS-key provided is correct, but simply return *invalidKey* every time. The timing delays continue to operate. This avoids revealing too much information to an attacker whilst still limiting the inconvenience caused by any legitimate errors during the unlock process.

Cycling the power to the ECU resets this behavior to allow further attempts. However, the timing sanctions imposed by *Delay_Timer* still apply from the start of the subsequent power cycle.

Key Generation

The RSA key-pairs must be generated using a high-quality cryptographic library and be of at least 3072 bits length. This provides a reasonable longevity to the key - NIST [9] states that this key-length provides comparable security to AES-128 and is suitable for “2030 and beyond” (in the absence of a cryptographically-relevant quantum computer).

ECU Key Provisioning

The verification keys are to be provisioned using the UDS Write Data By Identifier (0x2E) service. They must not be readable using any functions (including Read Data By Identifier (0x22), Read Memory By Address etc.) In order to verify that the keys have been correctly installed, a SHA-256 hash (*Verification_Keys_Hash*) of all the public keys is

available to be read via a UDS Data Identifier (DID). *Verification_Keys_Hash* is calculated from the flash storage of the verification keys at the time of request, to ensure that the actual storage is being checked. *Verification_Keys_Hash* can be checked at the end of the ECU production process to ensure the correct verification keys have been provisioned.

The verification-keys DID writing operations, and reading of *Verification_Keys_Hash*, are only accessible within one security level. This security level must not be used for any other operations. This security level can be accessed from a supplier specific session.

The default verification keys shall be initialized to an empty value (which should be the erased state of the flash memory area, all-bits-zero or all-bits-one, for example), except for the key-provisioning-key (the verification key for the DID-writing security level) which is provisioned and hard-coded into the software. Each verification key is only allowed to be written to by the Write Data By Identifier operation if it is currently empty. All other requests to write the verification keys are rejected with *RequestOutOfRange*. At the end of the production phase, after verifying the *Verification_Keys_Hash*, a service is invoked to write any unused verification-keys with a non-empty flag value (such as the inverse of the empty value). This marks them as unused and stops an attacker loading their own keys in the field.

When a *RequestSeed* message is received and the verification key for the requested security level holds the empty value, no UDS-Seed is generated. The ECU responds with *SubFunctionNotSupported*.

Cryptographic Details

In this section, comparison is made between the proposed scheme and that of an IT industry standard challenge-response mechanism. There are some particular features of the environment which are discussed.

One is that the tester does not need to authenticate the ECU. The reader will recall, from the introduction, that an attacker with physical access to the tester is expected to be able to extract the key material in some way. Indeed, a skilled attacker with access to the tester is expected to be able to bypass ECU authentication checks. Emulating an ECU to generate attacker-chosen UDS-Seed values might be one such attack.

Secondly, the applicable keys are explicitly known to both sides of the transaction, so key-spoofing attacks are out of scope. The only “identity” involved is that of the security-level being unlocked, which is established through the *SeedRequest* service.

Thirdly, the ECU does not need to authenticate a specific user. The fact that they have access to the tester which has the right key is sufficient for authorization.

These aspects mean that many conventional IT-based authentication approaches are more complex than necessary for this application (requiring the exchange of usernames, certificates, use of timestamps, key lookups on servers etc).

The protocol in this paper is heavily based on the U2F protocol [10] The challenge-response protocol is presented in

a pictorial manner in [4] and this is the protocol that is employed in this paper. In our context, the relying party is the ECU and the tester takes the place of the U2F device. There is no additional “client” layer.

There are some additional defenses provided in U2F which are discussed next.

U2F includes a defense against phishing attacks. In an IT context, phishing attacks redirect a user to a fake authentication site to retrieve their credentials. In this context, a fake ECU might be used to extract individual credentials from a tester. This would assume the tester is in the hands of the attacker - in our attacker model, we note that we do not expect to defend against such an attacker.

The defense against MITM attacks is not required as the connection is local and non-networked, so an attacker who can sit between a legitimate user and the ECU is likely to be noticed.

The next additional defense U2F provides is that of application-specific keys - we provide security-level and ECU-type-specific keys, which is merely a change in terminology.

U2F also defends against U2F-device cloning - this is not applicable in our case as all the testers (of a particular type) are clones of each other by design: they all contain the same key material, so it is not possible to differentiate authorized and unauthorized clones.

Finally, the ability to perform device attestation (the verification of attributes of the tester) is not something that the ECU needs to be able to perform.

The signature and verification operations are performed according to RSASSA-PSS [11].

Threat Analysis

A model-based TARA (according to ISO/SAE 21434 [12]) has been carried out on the design contained within this document. Initially, this modelled the “base-state of UDS unlock” which has been used in products like this to date. Additional controls and related assets were then added to the model to mitigate attacks on that base system, and the process iterated.

The appendix contains a block diagram showing the details of the key transfers along with a list of the residual risks within the design and the controls which were introduced to reach this stage.

The model captured very high-level outcomes for the damage scenarios. We modelled the impact of VM tuning with Moderate Operational impact, as compared to that of the ECU-design-entity tuning and code download, which could have Severe Safety impact. This leads to the separation of security levels and the use of multiple key-pairs to manage these accesses, following the standard security principle of least-privilege.

Attacks on the various infrastructure elements have been defended against. The reader will note that the ISO/SAE 21434 risk level has not changed (the Attack Feasibility was already assessed to be very low for the relevant attacks and so the pre-control risks were level 1). However, the attacks potentially affect a wide variety of locations. The defence of checking the

hash of the public keys before the end of production is relatively easy to implement, and so this has been included. It is interesting to observe from this that threat analysis cannot be performed as a purely numerical exercise.

It is common for legacy UDS unlock schemes to use very short UDS-Seeds and UDS-Keys [13], which presents the possibility of attackers being able to generate databases of the correct responses, or potentially even to perform a brute-force attack on the UDS-Key. The implemented UDS-Seeds and UDS-Keys have a length of 256-bits and thus form a useful control against such an attack. Legacy implementations have sometimes used a simple timer to provide the UDS-Seeds, meaning that an attacker can actually control the expected values from the UDS Seed Request, significantly limiting the size of the database they would require. In this design a more robust method of generating ECU-specific and time-varying entropy is used to seed a cryptographically strong random number generator - this will make it much more difficult for an attacker to build such a database, and also limit its application from one ECU to another.

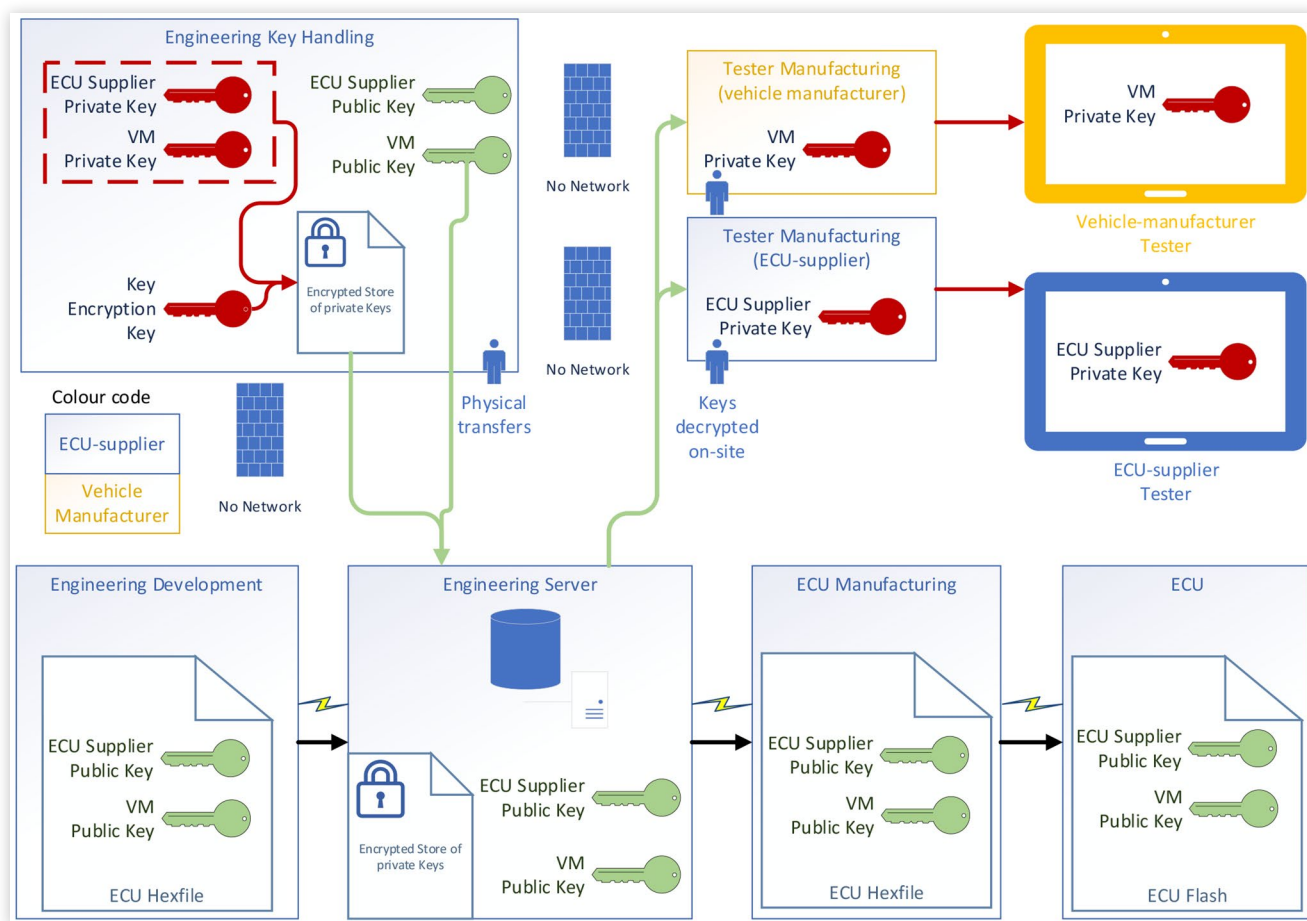
The use of public-key cryptography provided the means to limit the number of places where the private keys are exposed. Symmetric keys were modelled in an earlier iteration, and these must be defended on the ECU production line as well as in the tester and its production path. With asymmetric cryptography, although the tester path remains, the attacks to the production line are more limited. The private keys are defended, in this design, against network intrusions by only handling them in isolated environments and transferring them only in encrypted form, with the key-encryption-key stored and transferred physically, rather over the network.

Of course, the software content of the ECU must be protected against modification, requiring that any debugging interfaces are locked and that software downloads are authenticated. The latter provides defence-in-depth, over and above this design-pattern when it is used to defend the UDS code download interface.

The private keys used in the testers to authorize access are the most widely dispersed and powerful keys. As illustrated in Figure 2, these should be generated on a non-networked workstation and locally encrypted before they are transferred physically to the engineering server for long-term storage. Similarly, they are transferred physically to, and decrypted locally on, the non-networked tester-manufacturing-areas. These protections are particularly important for the most powerful (ECU-supplier) keys.

Summary and Future Work

This paper describes in detail a practical UDS Security Access design pattern which is applicable to a wide variety of ECUs. The initial (weakly defended) system was subject to many attacks, most of which can be mitigated to produce a much lower level of risk.

FIGURE 2 block diagram of the key interactions

A threat analysis has been performed and can be used by implementors as part of their “whole ECU” threat analysis efforts.

A future implementation could extend this work to defend the tester keys more effectively. This could be achieved by provisioning them into a local Trusted Platform Module, within the tester, or to a smart card which could be inserted into the tester. This would make it more difficult for an attacker to extract the key and clone their own testers.

References

1. Zhu, D., Pritchard, E., Reddy Dadam, S., Kumar, V. et al., “Optimization of Rule-Based Energy Management Strategies for Hybrid Vehicles Using Dynamic Programming,” *Combustion Engines* 184 (2021).
2. ISO, “ISO 14229-1:2020 Road Vehicles—Unified Diagnostic Services (UDS)—Part 1: Application Layer,” 2020, <https://www.iso.org/standard/72439.html>.
3. Wikipedia, “Challenge-Response Authentication,” https://en.wikipedia.org/wiki/Challenge%20%80%93response_authentication.
4. Yubico, “U2F Technical Overview,” accessed September, 2021, https://developers.yubico.com/U2F/Protocol_details/Overview.html.
5. NIST, “FIPS180-4—Secure Hash Standard (SHS),” 2015, <https://csrc.nist.gov/publications/detail/fips/180/4/final>.
6. NIST, “SP 800-90A—Recommendation for Random Number Generation Using Deterministic Random Bit Generators,” 2015, <https://csrc.nist.gov/publications/detail/sp/800-90a/rev-1/final>.
7. Microsoft, “Bitlocker Overview,” accessed August 2021, <https://docs.microsoft.com/en-us/windows/security/information-protection/bitlocker/bitlocker-overview>.
8. Basel Committee on Banking Supervision, “Core Principles for Effective Banking Supervision,” December 15, 2019, accessed October 2021, https://www.bis.org/basel_framework/chapter/BCP/01.htm?inforce=20191215&publish_ed=20191215.
9. NIST, “SP800-57 Recommendation for Key Management (rev5),” 2020.
10. FIDO Alliance, 2017, Accessed September 2021, <https://fidoalliance.org/specs/fido-u2f-v1.2-ps-20170411/fido-u2f-overview-v1.2-ps-20170411.pdf>.
11. IETF, “PKCS #1: RSA Cryptography Specifications Version 2.2,” November 2016, accessed November 2019, <https://tools.ietf.org/html/rfc8017>.

12. IEEE/SAE, "ISO/SAE 21434 Road Vehicles—Cybersecurity Engineering," 2021, accessed October 2021, <https://www.iso.org/standard/70918.html>.
13. Garcia, F. and Van den Herrewegen, J., "Beneath the Bonnet: A Breakdown of Diagnostic Security," 2018, Accessed October 2021, <https://www.cs.bham.ac.uk/~garcia/publications/BtB.pdf>.
14. NIST, "SP 800-38D—Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>.

Contact Information

Martin Thompson
martin.j.thompson@zf.com

Acknowledgments

The author is very grateful to his colleagues for their help with the development of this protocol and in reviewing this paper:

Andrew Bold, Michael Feiri, Gurchetan Grewal, Thorsten Hestermeyer. In addition, the author thanks the anonymous reviewers for their valuable input. Any remaining errors and omissions, of course, remain the author's.

Definitions/Abbreviations

CAN - Controller Area Network

CSPRNG - Cryptographically Strong Pseudo Random Number Generator

DID - Data Identifier

ECU - Electronic Control Unit

HSM - Hardware Security Module

TARA - Threat Analysis and Risk Assessment

TRNG - True Random Number Generator

UDS - Unified Diagnostics Services

VM - Vehicle Manufacturer

Appendix

Block Diagram

(Block Diagram.png is supplied separately in high resolution)

Residual Risks

Risk	Risk level with no controls	Residual Risk Level after controls	Risk Treatment	Controls
R.1: Tampering - ECU and its environment	5	3	Reduction	C.1, C.2, C.6
R.3: Information Disclosure - ECU supplier Tester	4	3	Reduction	C.11
R.4: Attacks using the tester messages	5	1	Reduction	C.3, C.4, C.5
R.5: Elevation of privilege within ECU	5	1	Reduction	C.7
R.6: Attacks on Production Line	1	1	Reduction	C.9
R.7: Attacks on Engineering Servers	1	1	Reduction	C.9
R.8: Attacks on development stations	1	1	Reduction	C.9
R.9: Information Disclosure - VM Tester	2	2	Acceptance	No further reasonably practical mitigations possible
R.10: Attacks on Key Handling stations	4	3	Reduction	C.10
R.11: Information Disclosure of the Key Encryption Key	4	3	Reduction	C.11

Controls

Name	Title	Description
C.1	Lock JTAG port	Lock the JTAG port with password or other disabling mechanism.
C.2	Software download authentication	Authenticate all software which is downloaded to the ECU before it is executed.
C.3	Public Key Signature - Security Algorithm	Use cryptographic methods for mapping UDS Seed to UDS Key
C.4	Rate limiting - Request	Limit the rate at which UDS requests relating to security are responded to
C.5	Length of UDS Seed 256-bits	In order to mitigate brute force extraction of the config from the tester, use a very long UDS Seed
C.6	High quality Entropy - UDSSeed	Ensure that the UDS Seed is generated from a good source of entropy
C.7	Well separated security levels	By providing different security levels, with different algorithm configurations, to cover different levels of privilege, the access to the configurations can be used to control access to functions. It is important to not distribute the keys more widely than necessary. Do not simply disallow access to the keys within the tester as this is likely to be easy to overcome.
C.8	Removed	C.8 was present in an earlier iteration of the design and is no longer required.
C.9	Check hash of security configuration	Testing Hash of the security configuration at the end of production ensures it has not been tampered with between when it was generated and when the ECU was produced
C.10	Separation of network - Key handling stations	Perform tasks which involve development and manufacturing with plaintext private keys on an airgapped workstation. Only transfer the encrypted files to and from off this station physically. Apply the 4-eyes principle to key handling.
C.11	Limited Distribution - ECU Supplier Tester	Limit the number of people who have access to the ECU SupplierTester (code or item) in order to reduce the probability of keys leaking
C.12	Limited Distribution - Key encryption key	Ensure that the KEK only distributed to those who need to know. Do not store on standard IT equipment, or in the cloud. Do not transfer by email. Apply the 4-eyes principle to key handling.